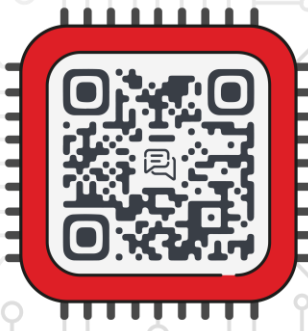




نظري

6

8

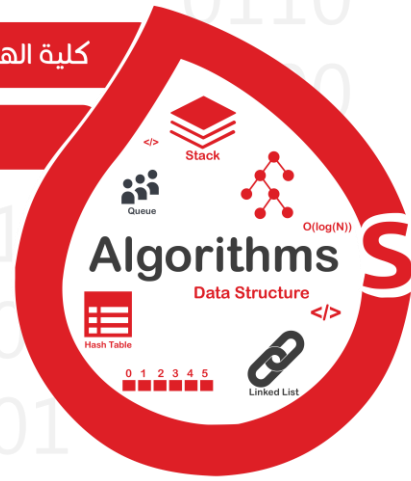
960 sp

كلية الهندسة المعلوماتية

السنة الثانية

Brute Force

د. أبي صندوق



**RB** Informatics; 28/12/2025

خوارزميات 1

كنا قد تحدّثنا سابقاً عن Decrease and Conquer و Divide and Conquer، و سنتابع اليوم مع النمط الثالث من أنماط التفكير و هو Brute Force (قوة مفرطة)، أيّ حلّ المسألة بالقوة، لأنّ هذه الطريقة تضمن إيجاد حلّ لأيّ مسألة قابلة للحلّ حاسوبياً لكن على حساب التعقيد.

## ما هو ال "Brute Force" ؟

هو أسلوب حل يعتمد على تجربة جميع الاحتمالات (الخيارات) الممكنة حتى يتم العثور على الحلّ الصحيح.

### مثال بسيط:

إذا أردنا معرفة فيما لو كانت مصفوفة أرقام تحتوي على عنصر يساوي قيمة معيّنة، نستخدم الطريقة التالية:

نبدأ من العنصر الأوّل و نفحصه، و إذا لم يكن هو العنصر المطلوب؛ ننتقل إلى العنصر الثاني، ثمّ الثالث،... و هكذا حتى نجد الرّقم (القيمة المطلوبة) أو تنتهي المصفوفة.

نستخدم Brute Force عند عدم القدرة على إيجاد طريقة حلّ أفضل أو عندما تكون المشكلة صغيرة الحجم بحيث يمكن تجربة كلّ الاحتمالات في وقت معقول، لأنّه عند ازدياد حجم البيانات يصبح استخدام خوارزمية Brute Force خياراً سيئاً لأنها بطيئة جداً عند التنفيذ -غير فعّالة- بسبب تزايد عدد الاحتمالات التي يجب تجربتها بشكل كبير جداً.

إذاً عند مواجهة مشكلة، نقوم بحساب عدد الخيارات المحتملة بشكل تقريبي، و النظر إلى عدد العمليات التي نحتاجها للتنفيذ، و بناءً عليه نقرّر استخدام هذه الخوارزمية أو اللّجوء إلى خوارزمية أخرى تقلّل من عدد العمليات، فإذا تأكّدنا من عدم وجود خوارزمية أخرى لحلّ المشكلة نعود إلى الحلّ عن طريق Brute Force.

بفرض لدينا قناة تلفزيونية تعرض برامج متنوعة وبين البرنامج الأول والبرنامج الثاني لدينا فراغ 90 ثانية نريد أن نملأه بإعلانات مختلفة (بحيث لا نكرر الإعلان نفسه أكثر من مرة ونملأ أكبر قدر ممكن من الفراغ الموجود)

## الإعلانات المتاحة :

| رقم الإعلان   | 1  | 2  | 3  | 4  | 5  | 6  |
|---------------|----|----|----|----|----|----|
| المدة الزمنية | 40 | 35 | 30 | 25 | 22 | 19 |

الحل:

نقوم بتجربة كل المجموعات الجزئية من هذه المجموعة

أي تجربة ( $2^6 = 64$ ) مجموعة جزئية

- ولا عنصر  $\frac{6!}{0!6!} = 1$  +
  - كل عنصر لوحده  $\frac{6!}{1!5!} = 6$  +
  - المجموعات الممكنة لعنصرين  $\frac{6!}{2!4!} = 15$  +
  - المجموعات الممكنة لثلاثة عناصر  $\frac{6!}{3!3!} = 20$  +
  - والمجموعات الممكنة لـ 6 عناصر  $\frac{6!}{6!0!} = 1$  +
- 64

استعملنا التوافق لحساب عدد المجموعات الممكنة

$$C = \frac{n!}{r!(n-r)!}$$

و ذلك لأن الترتيب بين الإعلانات غير مهم .

لنفرض أننا قررنا أن يكون الإعلان رقم 3 هو الإعلان الأول :

علينا تعبئة  
90 ثانية .

| 1  | 2  | 3  | 4  | 5  | 6  |
|----|----|----|----|----|----|
| 40 | 35 | 30 | 25 | 22 | 19 |

فتصبح الإعلانات المتاحة

علينا تعبئة  
60 ثانية .

| 1  | 2  | 4  | 5  | 6  |
|----|----|----|----|----|
| 40 | 35 | 25 | 22 | 19 |

ولنفرض أننا قررنا أن يكون الإعلان الثاني هو رقم 1 :

فتصبح الإعلانات المتاحة

علينا تعبئة  
20 ثانية .

| 2  | 4  | 5  | 6  |
|----|----|----|----|
| 35 | 25 | 22 | 19 |

× × × ✓

وبما أننا لا نستطيع تجاوز الـ 90 ثانية والإعلان رقم 2,4,5 لا يعتبر خياراً متاحاً ، لذا نختار الإعلان رقم 6 ونكون قد ملأنا 89 ثانية .

**ولكن** قد لا يكون هذا الجواب هو الأفضل نظراً لاختيارنا الإعلانات بشكل عشوائي ..

لنجرّب التراجع عن القرار الثاني وبدلاً من اختيار الإعلان رقم 1 سنختار الإعلان رقم 2 .  
فتصبح الإعلانات المتاحة

علينا تعبئة  
25 ثانية .

| 1  | 4  | 5  | 6  |
|----|----|----|----|
| 40 | 25 | 22 | 19 |

× ✓ ✓ ✓

ونكمل حتى نصل للجواب أو لطريق مسدود ...

وقد لا يكون أصلاً القرار الأول صحيح

## نستنتج :

لإيجاد الإجابة الصحيحة علينا تجربته جميع الإجابات الممكنة:

- نقوم بتثبيت قرار ونكمل المسألة على أساسه.
- ثم نتراجع عنه ونختار قرار آخر ونكمل المسألة على أساسه.
- وهكذا حتى نجد الجواب الصحيح.
- وبذلك نضمن أننا جربنا كل الإجابات الممكنة .



The Best view comes  
After , The Hardest climb.....

## التعقيد:

تتم هذه الطريقة بتعقيد زمني  $O(n!)$

حيث في البداية يكون لدينا 6 خيارات ثم 5 ثم 4 ثم 3 ثم 2 ثم 1 :

$$6 \times 5 \times 4 \times 3 \times 2 \times 1 = 6! \Rightarrow O(n!)$$

وهذا التعقيد مخفف عن التعقيد العام للـ Brute force في هذه المسألة ، حيث أنه في الحالة العامة يكون التكرار مسموحاً أي :

$$6 \times 6 \times 6 \times 6 \times 6 \times 6 = 6^6$$

و يكون التعقيد في الحالة العامة  $O(n^n)$  .

## ولكن

إن أعدنا التأمل نلاحظ أنه أحياناً يتم عرض الدعايات نفسها ولكن بترتيب مختلف وفي مسألتنا لا يهمنا الترتيب أي هي مجرد حالات متكافئة حسابها غير مجدٍ ومُكلف للوقت .

بالاعتماد على هذه الملاحظة :

يمكن اتباع آلية أخرى للتجريب :

بدلاً من أن نفكر (أي إعلان هو الأول بالترتيب) نفكر بكل إعلان هل سيعرض أم لا .

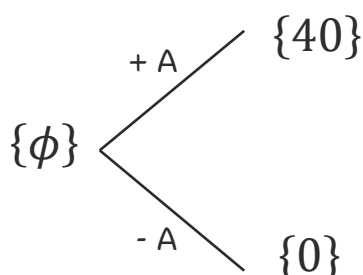
إذ أن المطلوب هو مجموع الثواني لا ترتيب الإعلانات .

وبما أن كل إعلان سيكون أمام خيارين ← يظهر ✓

← لا يظهر X

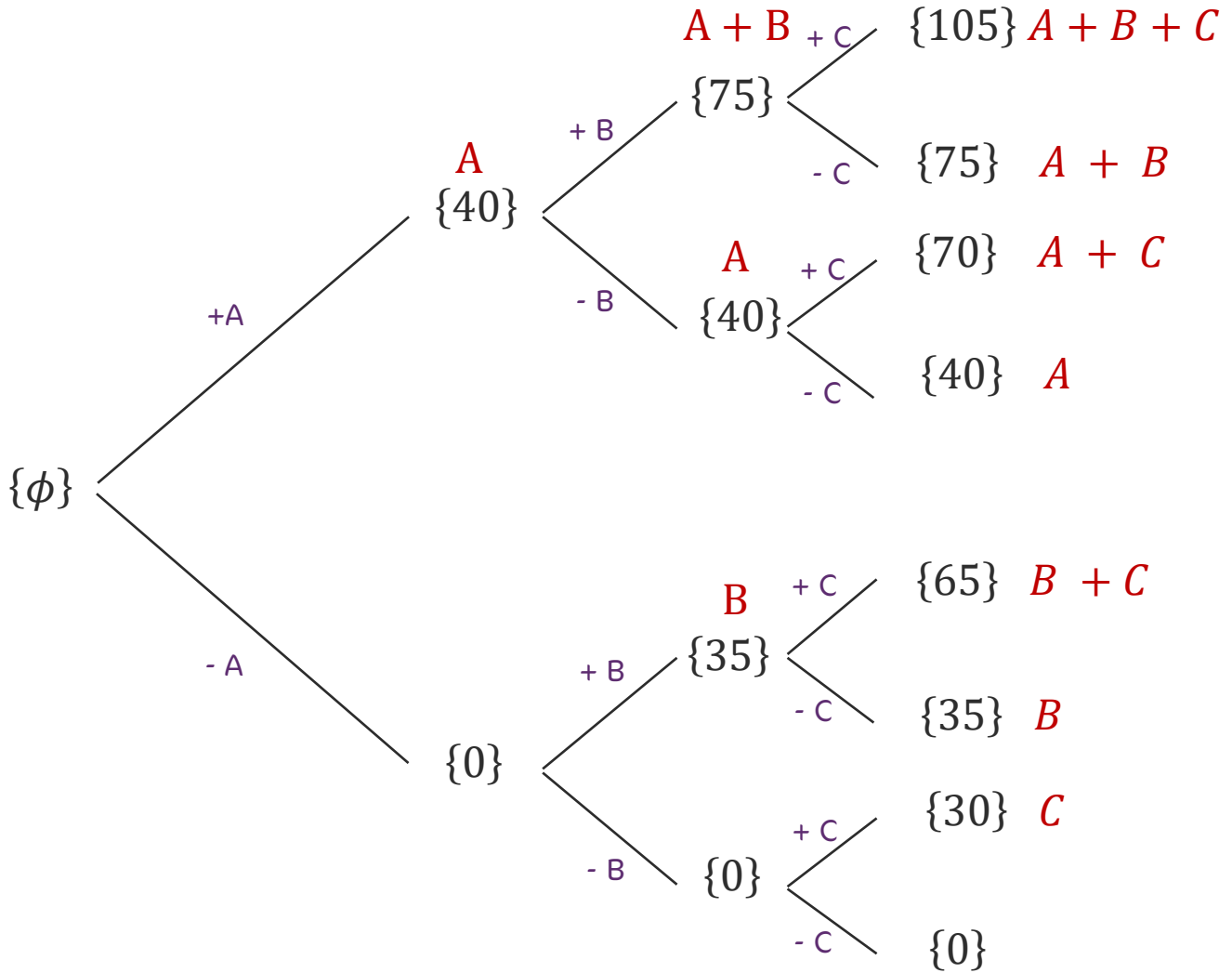
فالتعقيد الآن  $O(2^n)$  أي  $2^6$  .

| A  | B  | C  | D  | E  | F  |
|----|----|----|----|----|----|
| 40 | 35 | 30 | 25 | 22 | 19 |



قرار بأخذ الإعلان A قلل المدة الزمنية من 90 إلى 50

قرار بعدم أخذ الإعلان A قلل الإعلانات من 6 إلى 5



وصلنا إلى 8 مجموعات .

وتستمر الشجرة حتى نصل إلى 64 مجموعة

ملاحظة :

عندما وصلنا إلى المجموعة  $A+B+C$  كنا قد ملأنا 105 ثانية أي تجاوزنا الـ 90 ثانية بالفعل.  
وبالتالي يمكننا قطع الفرع من هنا دون أن نكمّله لأنه يستحيل إيجاد مجموعة حلها صحيح من إكماله.

إذاً في أسوأ الحالات ينتج لدينا 64 مجموعة وقد ينتج أقل من 64 إذا تمّ قصّ بعض الفروع.

## الخوارزمية :

شرح الكود:

السطر 1 : التابع الذي سنستدعيه للمرور على شجرة الاحتمالات.

السطر 2 : شرط التوقف أي إذا لم يبق خيارات (وصلنا لنهاية الفرع الذي نحن ضمنه) .

السطر 3 : التابع check optimum يختبر النتيجة التي حصلنا عليها بنهاية الفرع فإن كانت أفضل نتيجة يخزنها ، مثلاً: لو كانت آخر نتيجة مخزنه تغطي 80 ثانية:

- لو حصلنا بعدها على نتيجة تغطي 85 ثانية نحتفظ بها لأنها أفضل من 80 ثانية
- لو حصلنا بعدها على نتيجة تغطي 75 ثانية لا نحتفظ بها .

السطر 4 : وإلا (في حال وجود خيارات يمكن أخذها).

السطر 5 : من أجل كل قرار متاح.

السطر 6 : إذا كان قرار صالح (أي لم يتجاوز الـ 90 بعد).

السطر 7 : نقوم بحفظ الخيار (أي نعتبره جزءاً من الحل الحالي) .

السطر 8 : نعيد استدعاء التابع من أجل الخطوة التالية (أي من أجل الخيار التالي) .

السطر 9 : نزيل هذا القرار لنتخذ قراراً جديداً (أي نتراجع عن هذا القرار ونتخذ قرار جديد).

```

1. Try(int i){
2.     if(finalstate(i))
3.         check optimum();
4.     else{
5.         for each (option(v)){
6.             if(vaild option(v)){
7.                 store option(v);
8.                 Try(i+1);
9.                 Backtrack option(v);
10.            }
11.        }
12.    }
13.}

```

## ملاحظة:

Store Option(v) : أي بـ Data Structure ما .

1. خزن الإعلان A .

2. خزن عدد الثواني المتبقيه (50 هنا).

3. قم بإلغاء A من قائمة للخيارات المتاحة.

Backtrack Option(v) : أي اذهب لـ Data Structure نفسها .

1. قم بإلغاء تخزين الإعلان A .

2. أعد الثواني المحذوفة (عدنا إلى 90 ثانية هنا) .

3. أعد A إلى قائمة الخيارات المتاحة.

مثال عن Data Structure :

مصفوفة طولها بعدد الإعلانات

إن أخذنا الإعلان نضع 1 .

إن لم نأخذه نضع 0 .

الخوارزمية السابقة تمثل الشكل العام لـ Backtracking .

**التعقيد:**

وضوحاً التعقيد أسي

لكل إعلان لدينا خيارين أي  $O(2^n)$  .

لكن يمكن تخفيف التعقيد بالتلاعب بالشرط في السطر السادس بوضع شروط للاستمرار في الفرع .  
عندها ننتقل من الـ Backtracking (والتي تعني تجريب كل خيار مهما كان) ليصبح لدينا نوع خاص يدعى:

• Branch & Bound .

↳ Branch: فرع .

↳ Bound: قلم (أي قصّ الفروع غير الضرورية)

في السطر السادس في المثال السابق:

إذا تجاوزنا الـ 90 ثانية فلا داعي للإكمال في الفرع .

ويمكن إضافة شرط آخر مثلاً :

إذا كان مجموع الثواني من مكان وجودي بالفرع إلى نهايته لا يعطيني حلاً أفضل من حل موجود لدي سابقاً ،

فلا داعي للإكمال في الفرع .

• إن تعقيد خوارزمية Backtracking كان  $O(2^n)$  .

أما تعقيد خوارزمية Branch & Bound فهو أخف من  $O(2^n)$  ولكن يبقى ضمن الرتبة الأسية .

• نسبة التخفيف تعتمد على مدى وضوح قرارات الـ Bound وعددها .

في خوارزمتنا :

■ معامل التفرع (عند كل خيار كم فرع يتفرّع ؟) : 2 (a)

■ ما العمق المحتمل للشجرة ؟ كحد أقصى 6 (b)

■ عدد الخيارات الممكنة : كحد أقصى  $2^6$  (a<sup>b</sup>)

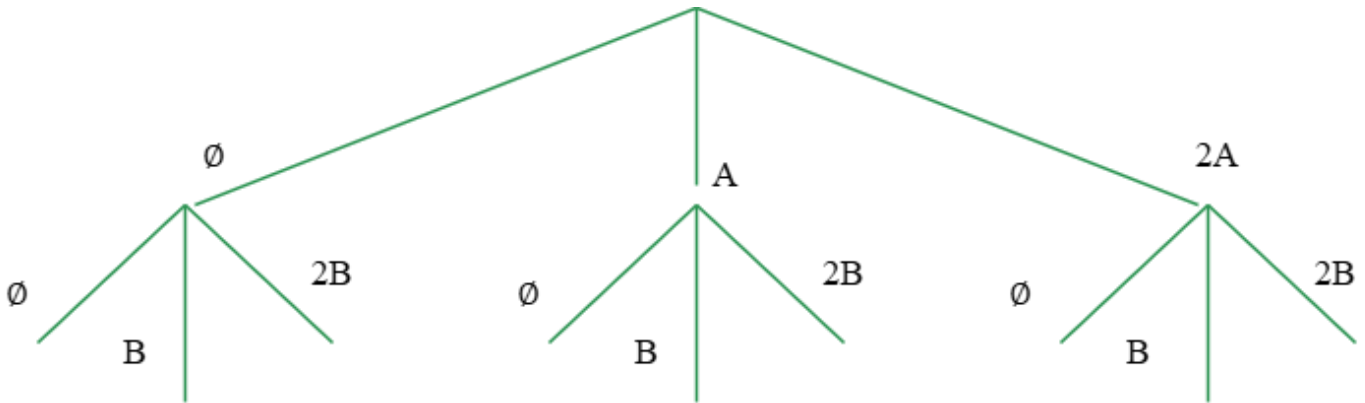
التعقيد قلّ من  $6^6$  إلى  $2^6$  لأننا انتقلنا من خوارزمية من هي الدعاية الأولى فالثانية إلى خوارزمية هل ستظهر هذه الدعاية أم لا . و بهذا نكون قد قللنا الـ Breadth / Branch (عدد القرارات التي يجب تجربتها) ويمكن تقليل الـ Depth عن طريق القصصنة (Branch & Bound).

## لنفترض أنه تم تغيير القيود على المسألة

مثلاً أن نسمح للإعلان أن يتكرر مرتين على الأكثر و بهذا الشكل يصبح لكل قرار ثلاث احتمالات:

- أن لا يأخذ الإعلان  $\emptyset$ .
- أن يأخذ الإعلان مرة واحدة  $+A$ .
- أن يأخذ الإعلان مرتين  $+2A$ .

بالتالي سيصبح التعقيد من الشكل:  $(3)^n$



أنت أقوى مما تظن،

و مستقبلك مشرق بإذن الله...

